

```
In [50]: import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans
import warnings
warnings.filterwarnings('ignore')
```

```
In [5]: df = pd.read_csv('E:/Data analytics learning/data analysis project/Python - Custom
```

```
In [3]: df.head()
```

```
Out[3]:
```

|   | CustomerID | Gender | Age | Annual Income (k\$) | Spending Score (1-100) |
|---|------------|--------|-----|---------------------|------------------------|
| 0 | 1          | Male   | 19  | 15                  | 39                     |
| 1 | 2          | Male   | 21  | 15                  | 81                     |
| 2 | 3          | Female | 20  | 16                  | 6                      |
| 3 | 4          | Female | 23  | 16                  | 77                     |
| 4 | 5          | Female | 31  | 17                  | 40                     |

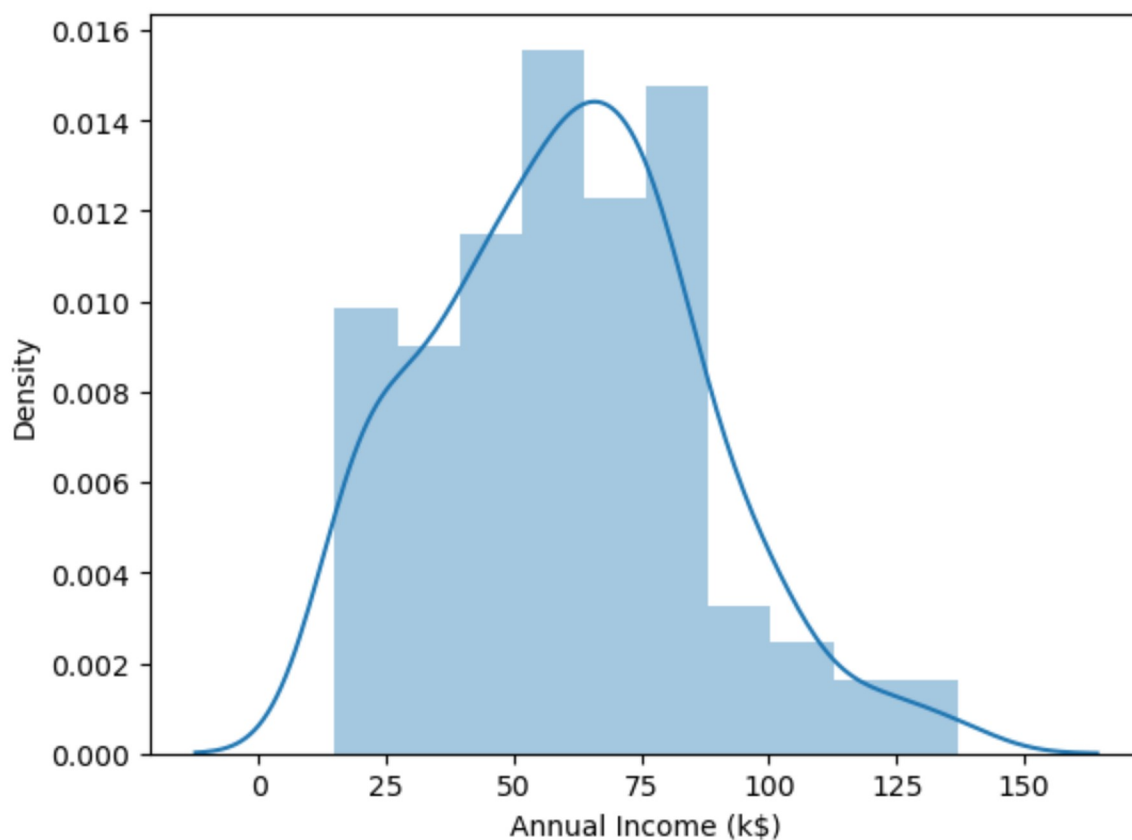
## Univariate Analysis

```
In [4]: df.describe()
```

```
Out[4]:
```

|       | CustomerID | Age        | Annual Income (k\$) | Spending Score (1-100) |
|-------|------------|------------|---------------------|------------------------|
| count | 200.000000 | 200.000000 | 200.000000          | 200.000000             |
| mean  | 100.500000 | 38.850000  | 60.560000           | 50.200000              |
| std   | 57.879185  | 13.969007  | 26.264721           | 25.823522              |
| min   | 1.000000   | 18.000000  | 15.000000           | 1.000000               |
| 25%   | 50.750000  | 28.750000  | 41.500000           | 34.750000              |
| 50%   | 100.500000 | 36.000000  | 61.500000           | 50.000000              |
| 75%   | 150.250000 | 49.000000  | 78.000000           | 73.000000              |
| max   | 200.000000 | 70.000000  | 137.000000          | 99.000000              |

```
In [6]: sns.distplot(df['Annual Income (k$)']);
```

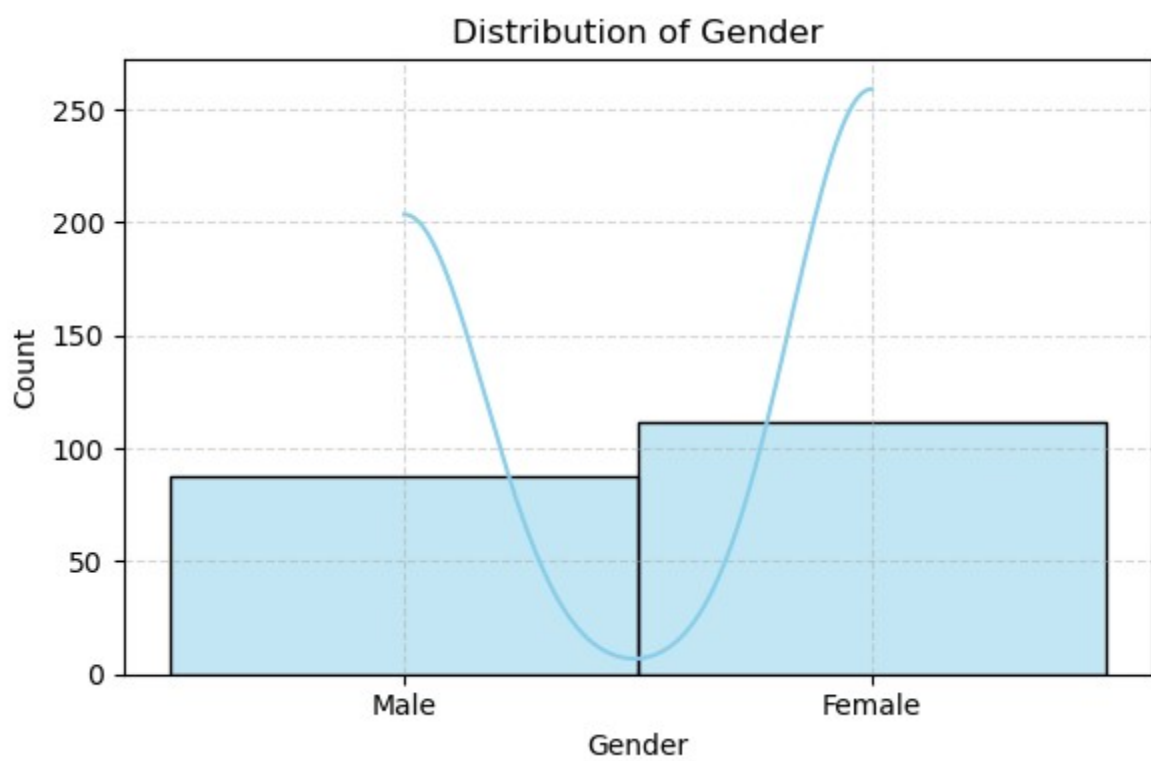
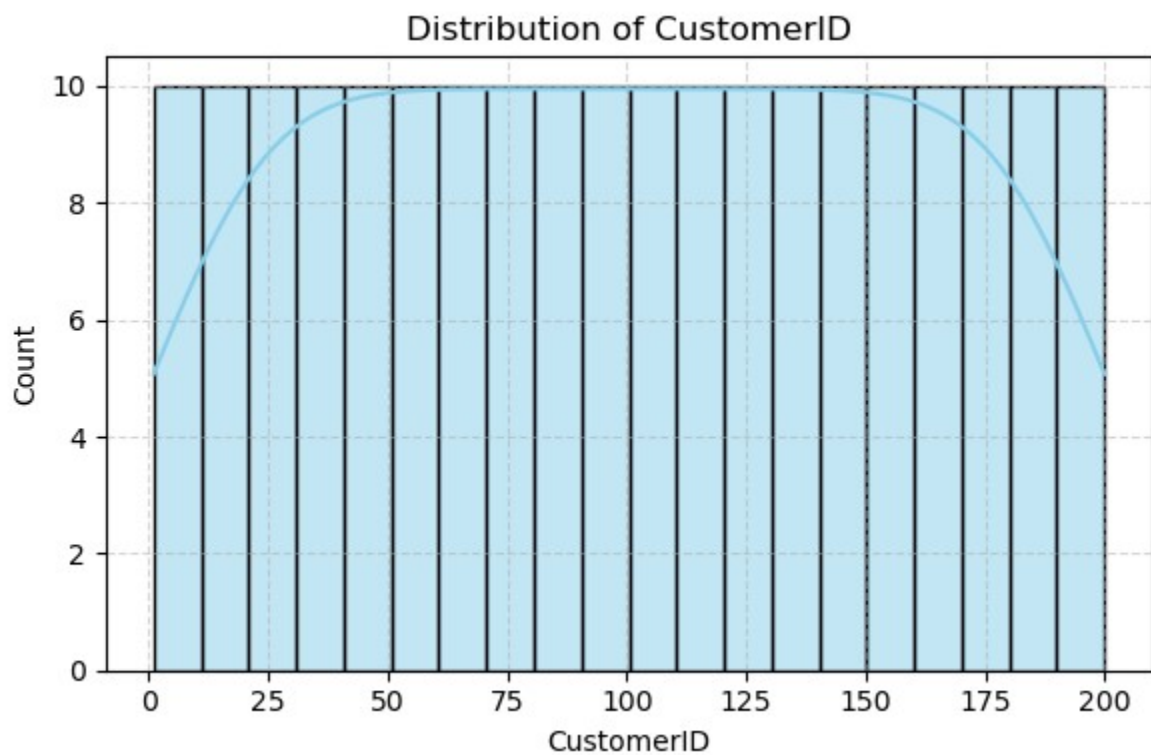


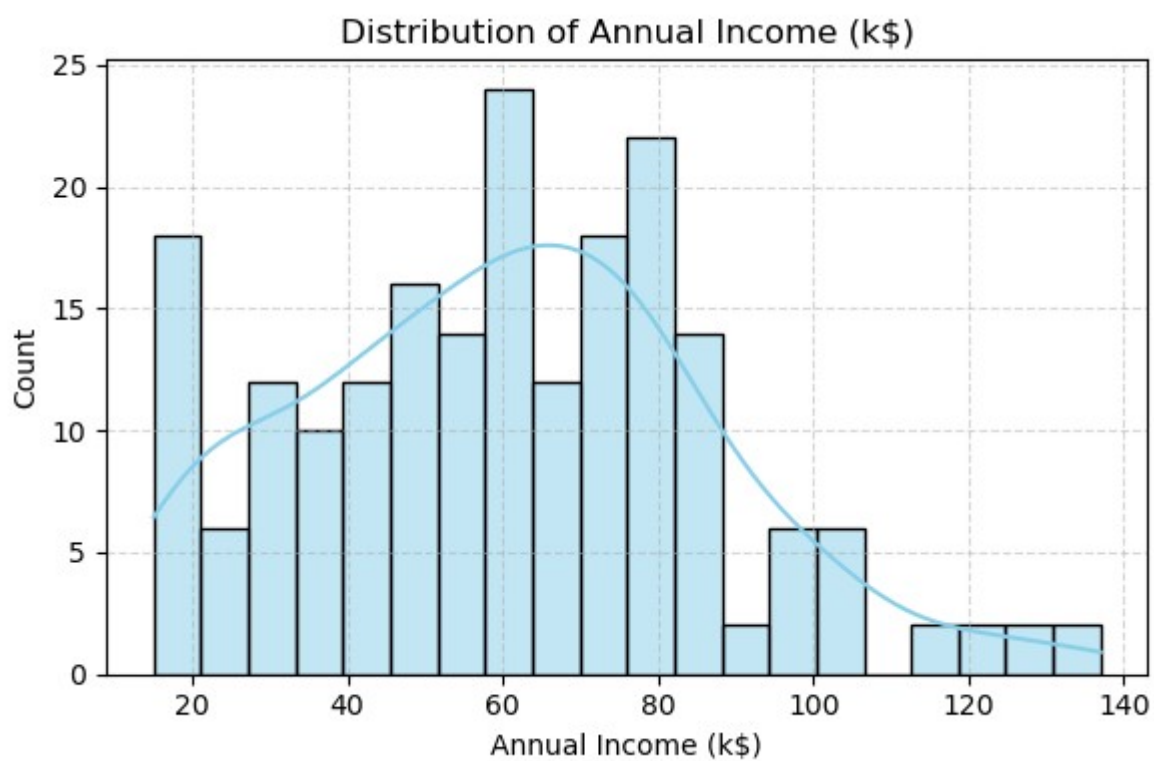
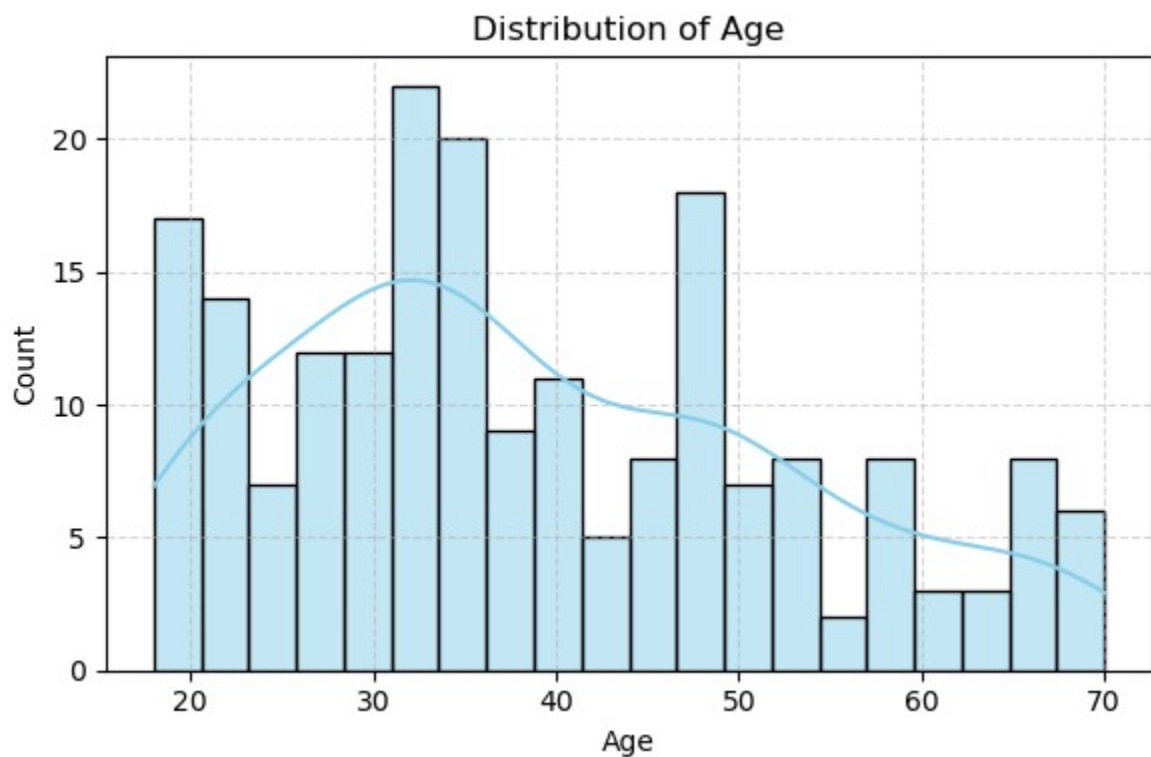
In [7]: `df.columns`

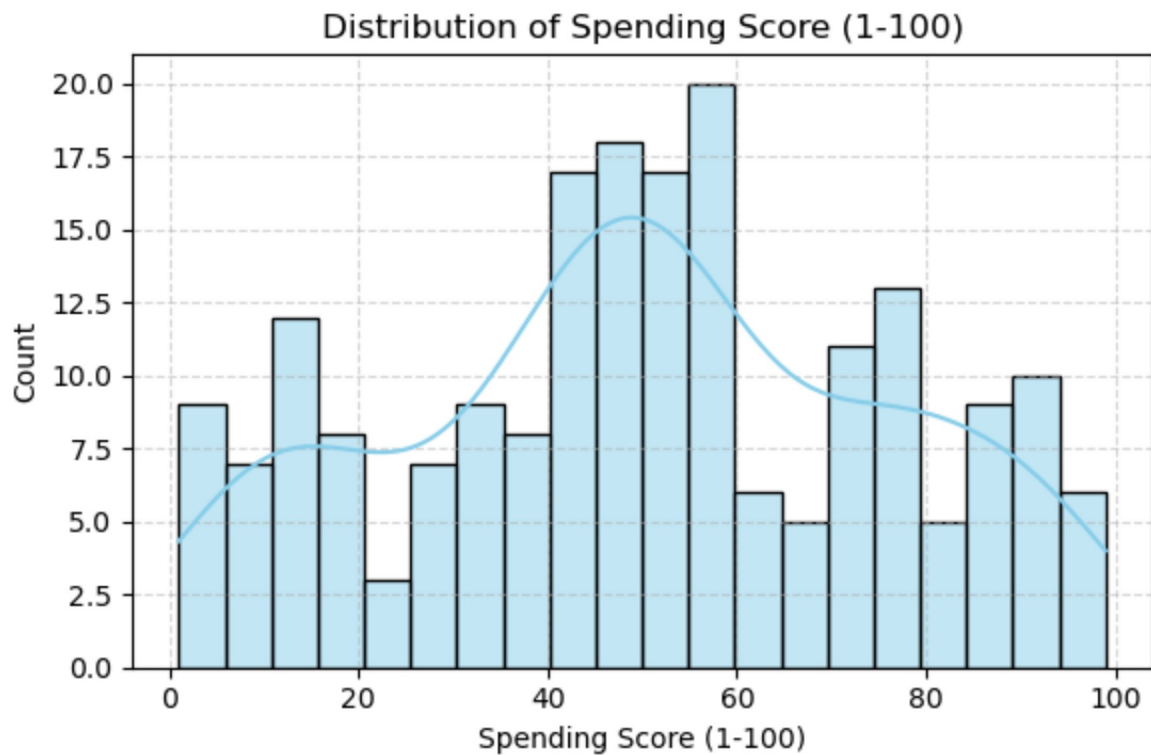
Out[7]: Index(['CustomerID', 'Gender', 'Age', 'Annual Income (k\$)',  
'Spending Score (1-100)'],  
dtype='object')

```
In [13]: # List of columns you want to visualize
columns = ['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)']

# Loop through each column and create a separate histogram
for i in columns:
    plt.figure(figsize=(6,4)) # Create a new figure for each column with fixed size
    sns.histplot(df[i], kde=True, bins=20, color='skyblue') # Plot histogram + KDE
    plt.title(f"Distribution of {i}", fontsize=12) # Add title for each histogram
    plt.xlabel(i) # Label x-axis with the column name
    plt.ylabel("Count") # Label y-axis as count
    plt.grid(True, linestyle='--', alpha=0.5) # Add light gridlines for readability
    plt.tight_layout() # Adjust layout to prevent label overlap
    plt.show() # Display the plot before moving to the next one
```



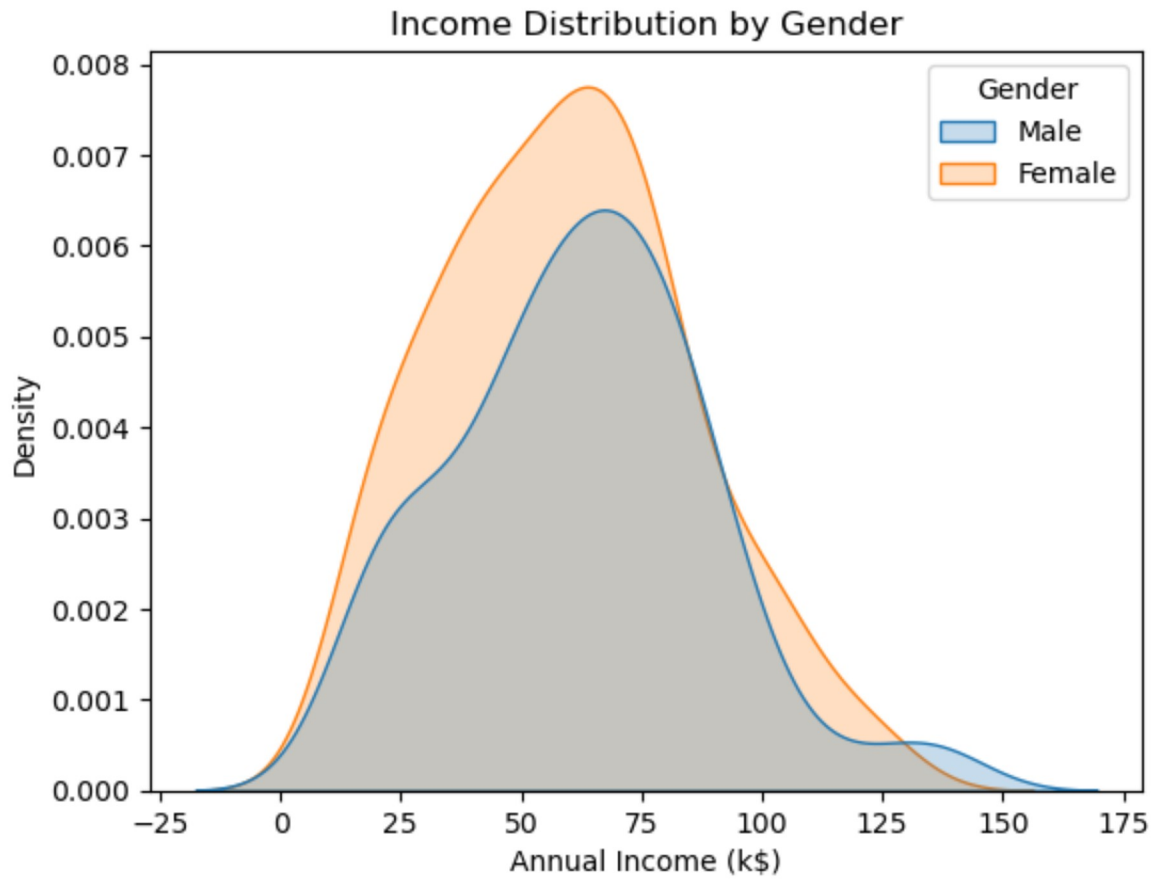




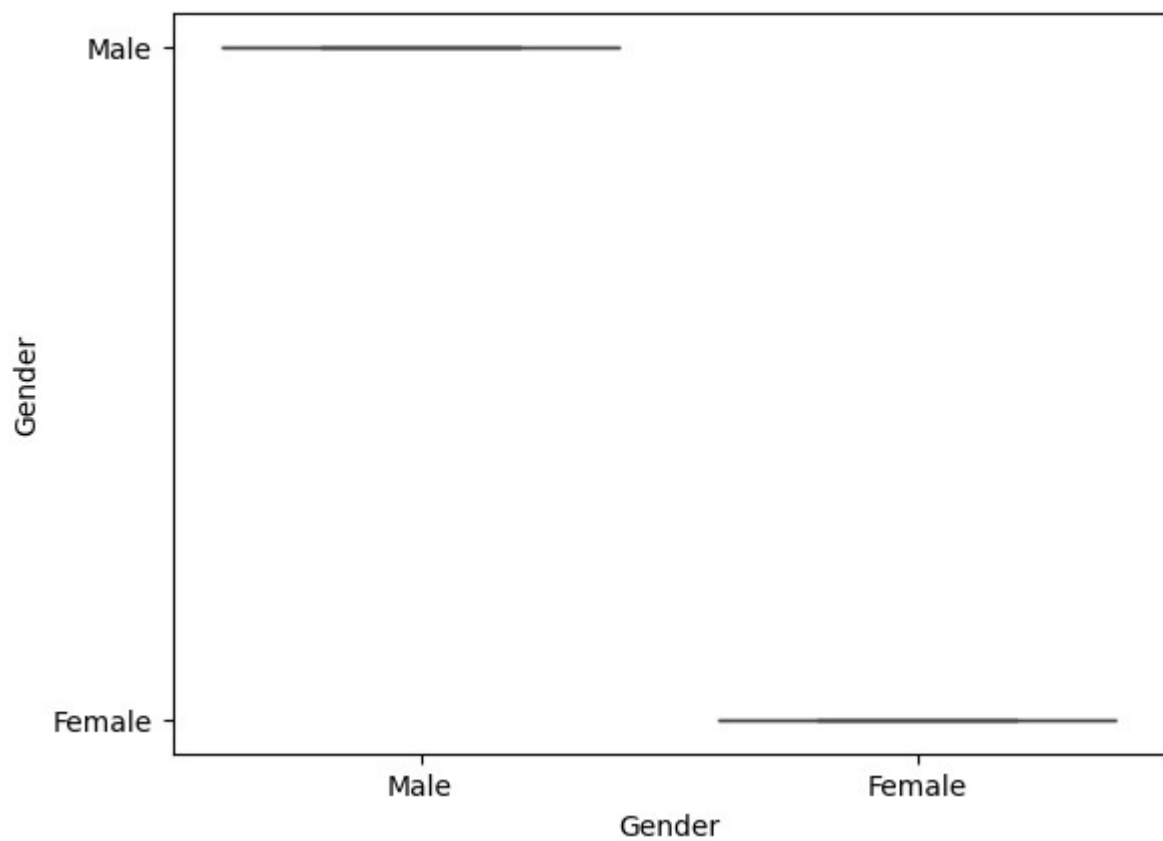
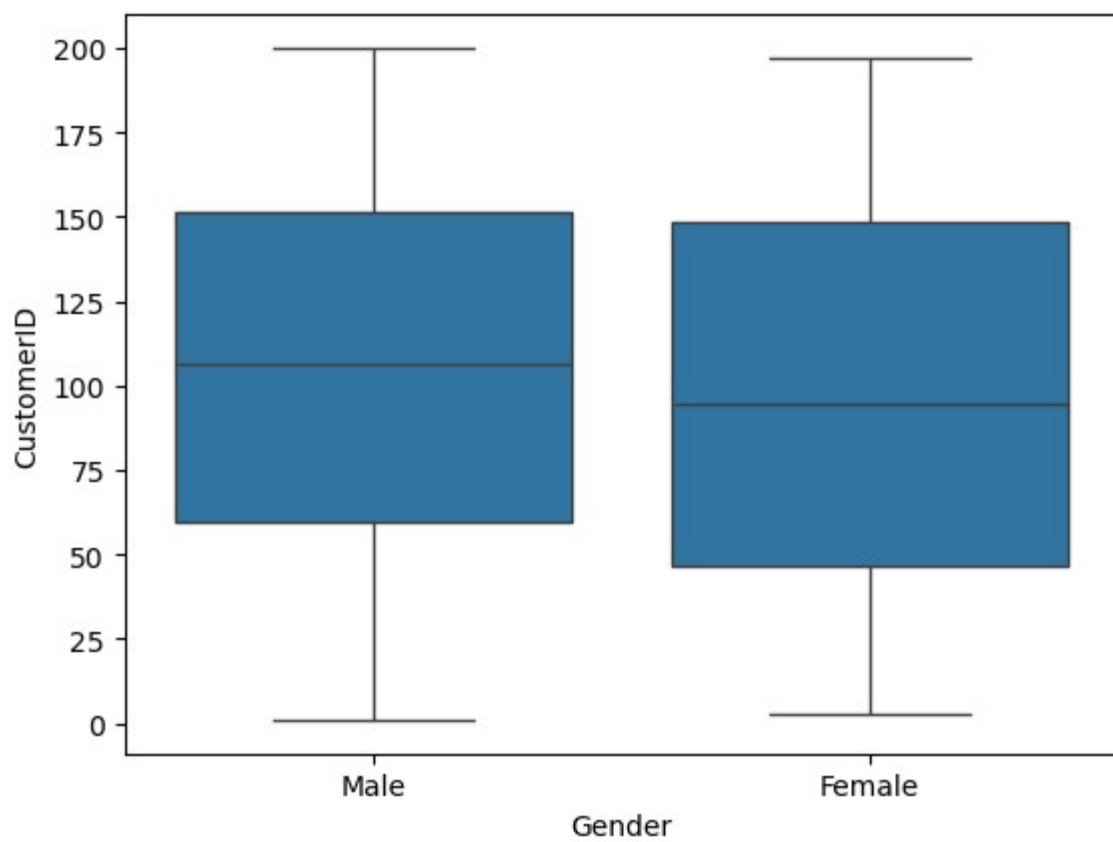
```
In [16]: # Plot a Kernel Density Estimate (KDE) of 'Annual Income (k$)' for each gender
sns.kdeplot(data=df, x='Annual Income (k$)', hue='Gender', fill=True)

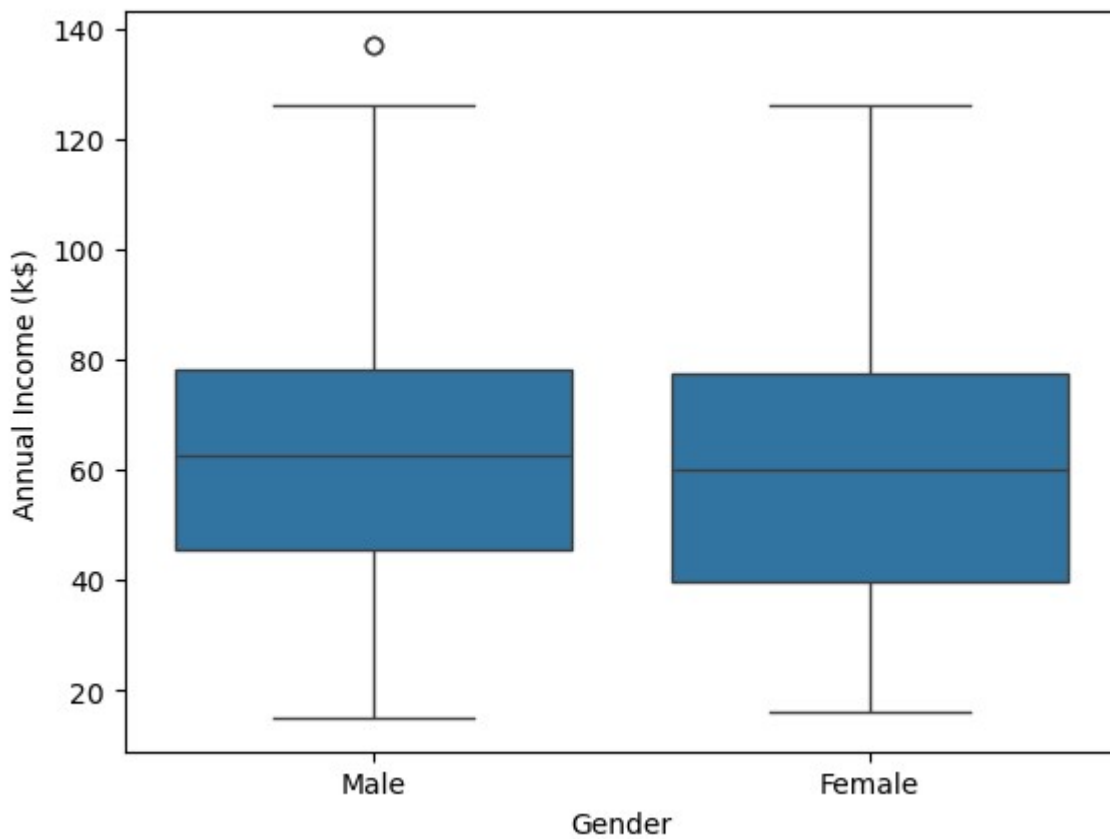
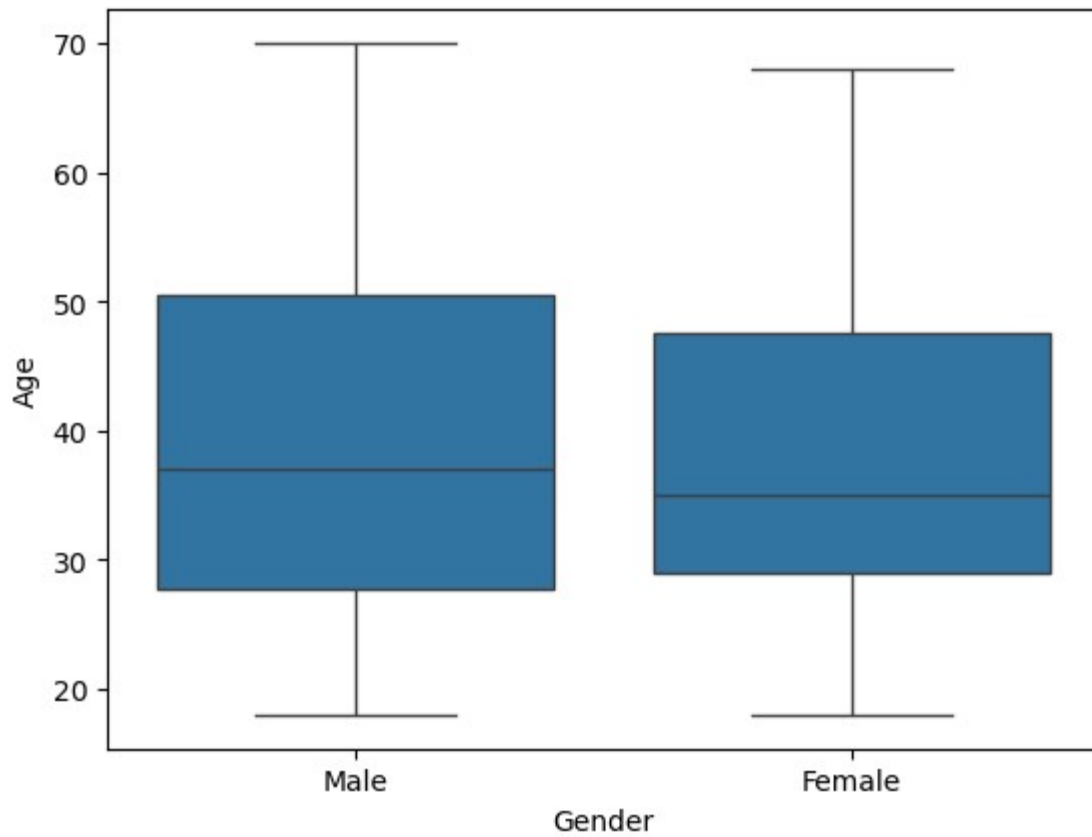
# Add a title to the plot
plt.title("Income Distribution by Gender")

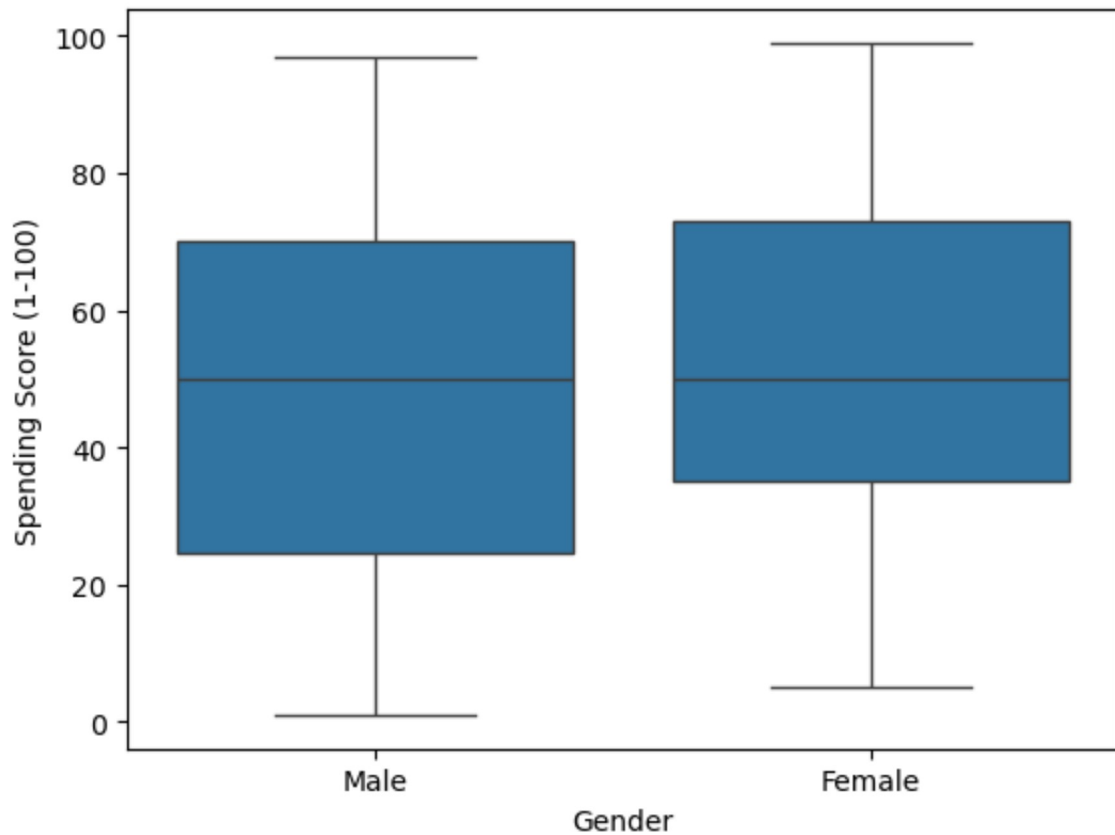
# Display the plot on the screen
plt.show()
```



```
In [17]: columns = ['CustomerID', 'Gender', 'Age', 'Annual Income (k$)', 'Spending Score (1-  
  
for i in columns: # Loop through each column in the list  
    plt.figure() # Create a new figure for each boxplot  
    sns.boxplot(data=df, x='Gender', y=df[i]) # Draw a boxplot comparing the column
```





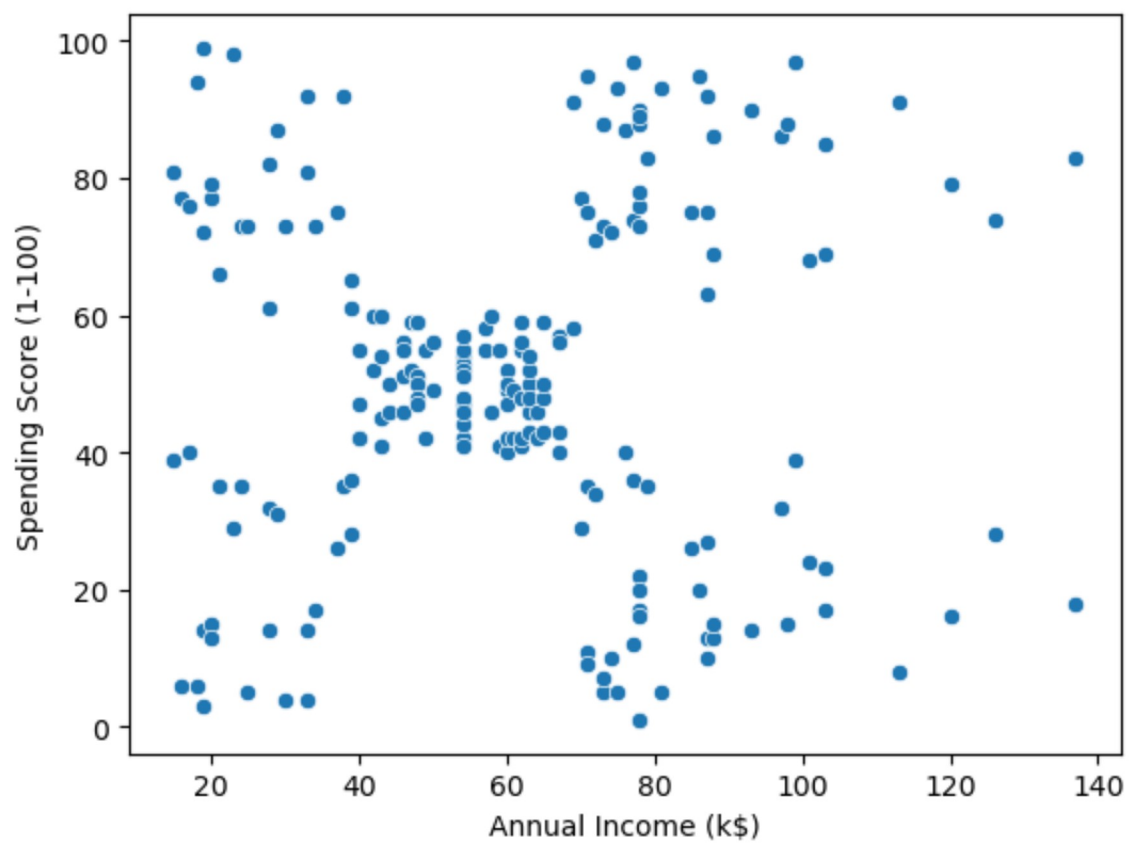


```
In [7]: df['Gender'].value_counts()
```

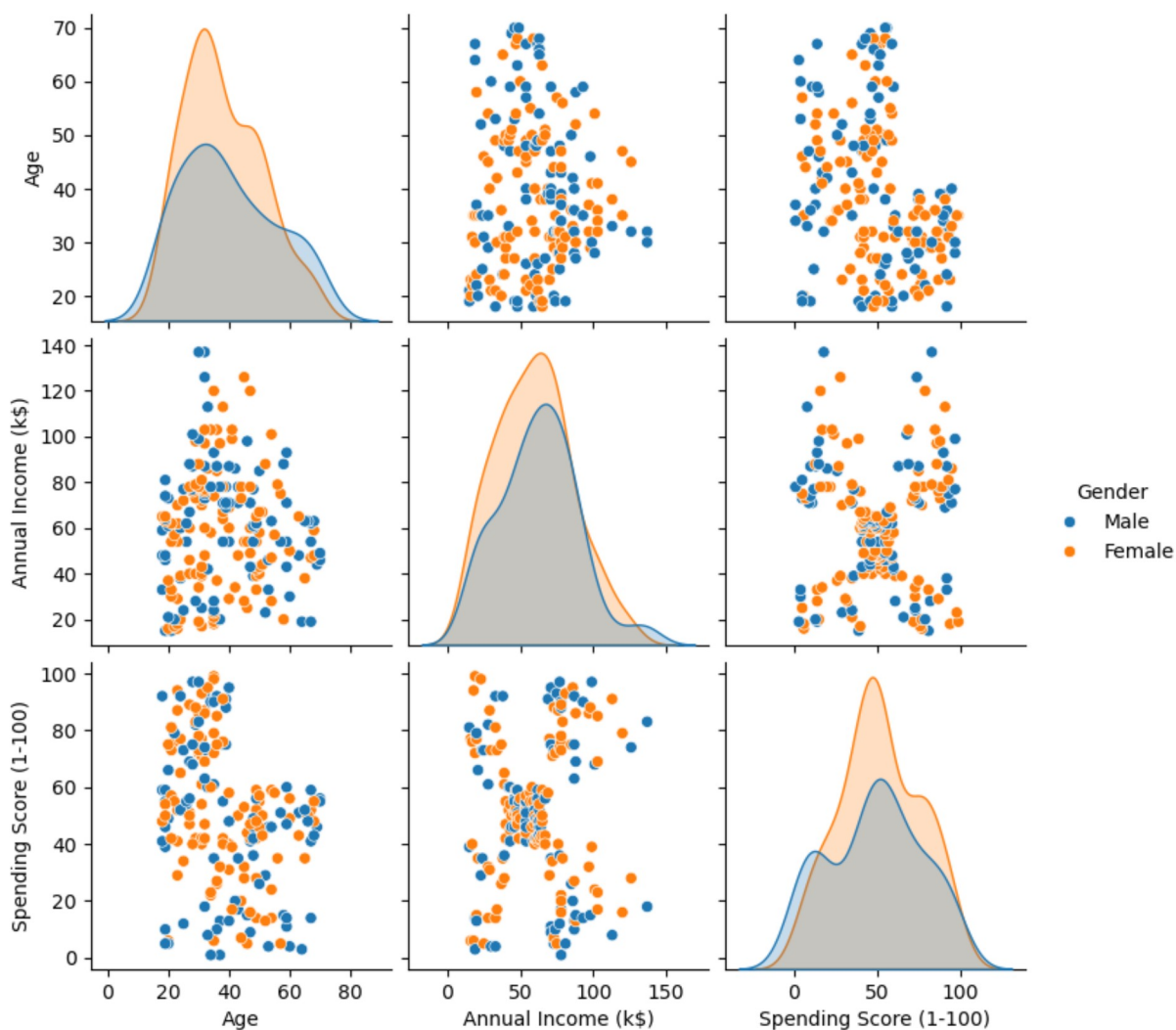
```
Out[7]: Gender
Female    112
Male      88
Name: count, dtype: int64
```

## Bivariate Analysis

```
In [6]: sns.scatterplot(data=df, x='Annual Income (k$)', y='Spending Score (1-100)');
```



```
In [10]: # df = df.drop('CustomerID',axis = 1)
sns.pairplot(df,hue = 'Gender');
```



```
In [17]: df.groupby('Gender')[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].mean()
```

```
Out[17]:
```

|        | Age       | Annual Income (k\$) | Spending Score (1-100) |
|--------|-----------|---------------------|------------------------|
| Gender |           |                     |                        |
| Female | 38.098214 | 59.250000           | 51.526786              |
| Male   | 39.806818 | 62.227273           | 48.511364              |

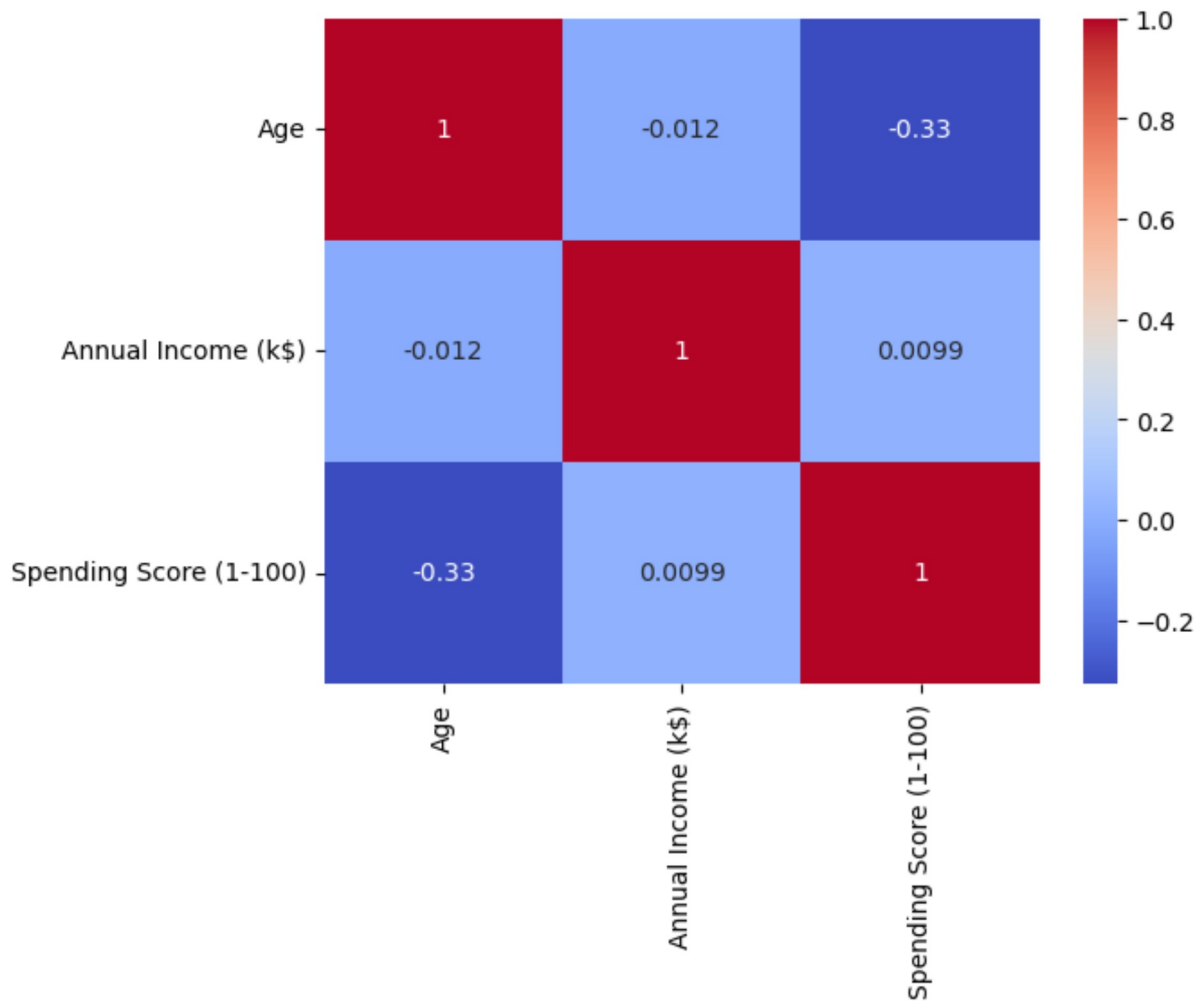
```
In [20]: df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].corr() # give correlat
```

```
Out[20]:
```

|                        | Age       | Annual Income (k\$) | Spending Score (1-100) |
|------------------------|-----------|---------------------|------------------------|
| Age                    | 1.000000  | -0.012398           | -0.327227              |
| Annual Income (k\$)    | -0.012398 | 1.000000            | 0.009903               |
| Spending Score (1-100) | -0.327227 | 0.009903            | 1.000000               |

```
In [23]: sns.heatmap(df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].corr(), ann
```

Out[23]: <Axes: >



## Clustering - univariate

```
In [38]: clustering1 = KMeans(n_clusters = 3) # Create a KMeans clustering model object using
```

```
In [39]: clustering1.fit(df[['Annual Income (k$)']]) # Train (fit) the KMeans model using t
```

```
Out[39]:  
▼ KMeans ⓘ ?  
KMeans(n_clusters=3)
```

```
In [40]: clustering1.labels_
```

```
Out[40]: array([1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 0], dtype=int32)
```

```
In [41]: df['Income Cluster'] = clustering1.labels_ # Create a new column in the DataFrame
df.head() # Display the first 5 rows of the DataFrame to check the new 'Income Clu
```

```
Out[41]:
```

|   | Gender | Age | Annual Income (k\$) | Spending Score (1-100) | Income Cluster |
|---|--------|-----|---------------------|------------------------|----------------|
| 0 | Male   | 19  | 15                  | 39                     | 1              |
| 1 | Male   | 21  | 15                  | 81                     | 1              |
| 2 | Female | 20  | 16                  | 6                      | 1              |
| 3 | Female | 23  | 16                  | 77                     | 1              |
| 4 | Female | 31  | 17                  | 40                     | 1              |

```
In [42]: df['Income Cluster'].value_counts()
```

```
Out[42]: Income Cluster
2      86
1      58
0      56
Name: count, dtype: int64
```

```
In [43]: clustering1.inertia_ # Returns the sum of squared distances of samples to their ne
```

```
Out[43]: 25640.45778439684
```

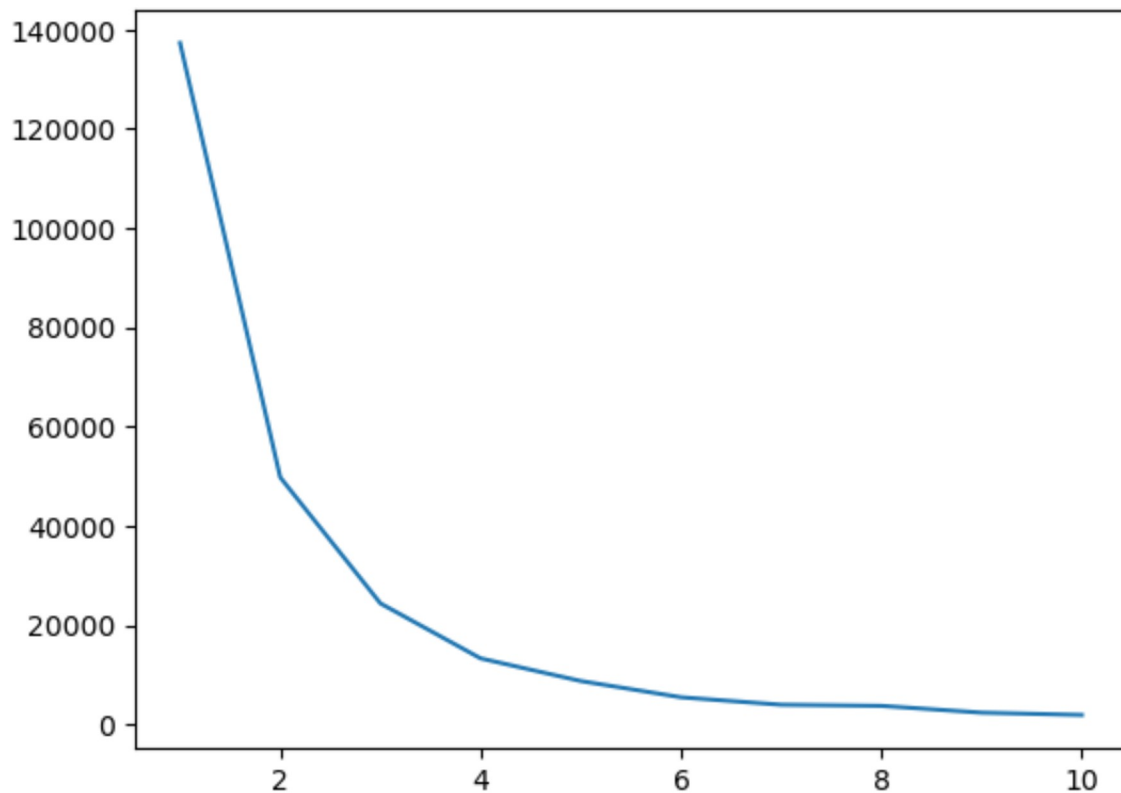
```
In [44]: inertia_scores = [] # Empty list to store inertia values for each number of cluste

for i in range(1, 11): # Loop through cluster numbers from 1 to 10
    kmeans = KMeans(n_clusters=i) # Create a KMeans model with 'i' clusters
    kmeans.fit(df[['Annual Income (k$)']]) # Fit the model on the 'Annual Income (
    inertia_scores.append(kmeans.inertia_) # Store the inertia (sum of squared dis
```

```
In [45]: inertia_scores
```

```
Out[45]: [137277.28000000006,  
          49761.73701298703,  
          24361.259213759207,  
          13278.112713472483,  
          8717.458097165994,  
          5443.614973544973,  
          3941.4163614163635,  
          3717.3905677655675,  
          2346.7116236704483,  
          1883.5716374269005]
```

```
In [46]: plt.plot(range(1,11),inertia_scores);
```



```
In [47]: df.columns
```

```
Out[47]: Index(['Gender', 'Age', 'Annual Income (k$)', 'Spending Score (1-100)',  
               'Income Cluster'],  
              dtype='object')
```

```
In [49]: df.groupby('Income Cluster')[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']  
# Groups the data by 'Income Cluster' and calculates the average Age, Income, and S
```

Out[49]:

|                | Age       | Annual Income (k\$) | Spending Score (1-100) |
|----------------|-----------|---------------------|------------------------|
| Income Cluster |           |                     |                        |
| 0              | 36.910714 | 92.142857           | 50.517857              |
| 1              | 37.120690 | 29.551724           | 49.689655              |
| 2              | 41.279070 | 60.906977           | 50.337209              |

```
In [ ]:
```